Лабораторная работа №4

«Обнаружение и выделение признаков на изображениях и видеокадрах для решения производственных задач»

Цель работы: приобрести навыки выделения и анализа признаков объектов на изображениях и видеокадрах (контуров, границ, ключевых точек, геометрических и цветовых характеристик), а также освоить один из алгоритмов сегментации изображения (заливка, пороговая бинаризация или метод Оцу) как этап, предшествующий разметке данных и обучению нейронной сети для решения производственных задач технического зрения.

Теоретические сведения

Прежде чем передавать изображения в модель машинного обучения, необходимо понять, какие именно признаки объекта являются информативными для решаемой задачи. Признаком называется измеримая характеристика изображения или его фрагмента, по значению которой можно отличить один объект (или класс объектов) от другого. К таким характеристикам относятся форма и контур объекта, его размеры, цвет и текстура поверхности, взаимное расположение частей, а также локальные особые точки – углы, перегибы контура, характерные пятна текстуры.

В зависимости от того, какая информация требуется от системы технического зрения, выделяют три основные задачи компьютерного зрения: классификация – определение того, к какому классу относится изображение целиком (например, «деталь без дефекта» / «деталь с дефектом»); детектирование – определение того, что и где на изображении находится, с указанием ограничивающей рамки (bounding box); сегментация – определение того, какие именно пиксели изображения относятся к объекту, с построением его маски.

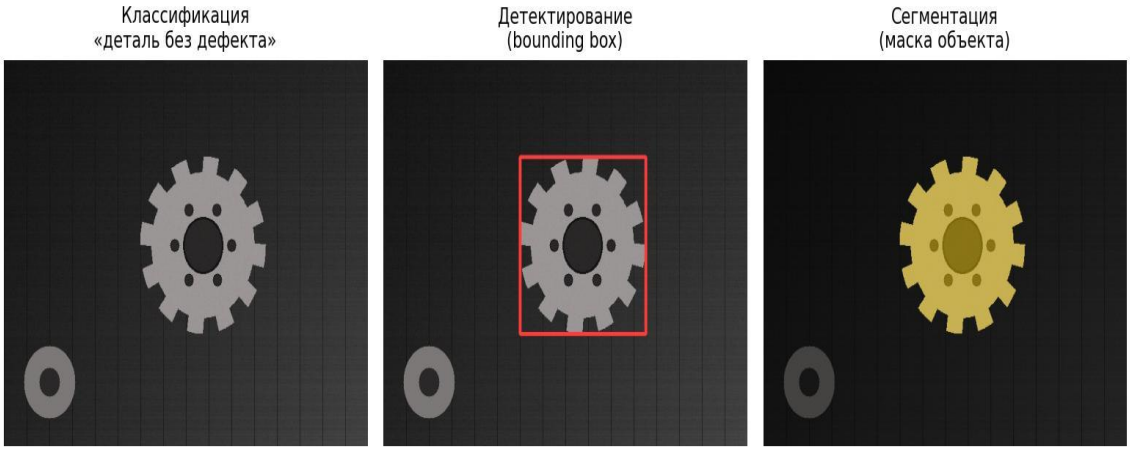


Рисунок 1 – Три основные задачи компьютерного зрения на примере детали «шестерня»

Выбор задачи определяет, какие признаки нужно выделять и в каком виде их подавать далее на разметку и обучение. Для классификации достаточно интегральных признаков всего изображения (гистограмма, текстура, средняя яркость), для детектирования требуется локализация объекта, а для сегментации – точное описание его границ.

Контуры и границы объектов

Граница объекта – это область изображения, в которой яркость или цвет резко меняются при переходе от объекта к фону или от одной детали объекта к другой. Выделение границ выполняется операторами дифференцирования яркости, наиболее распространённым из которых является детектор Кэнни (Canny). Он последовательно выполняет сглаживание изображения, вычисление градиента яркости по двум направлениям, подавление немаксимумов и пороговую фильтрацию по двум порогам (нижнему и верхнему), что позволяет получить тонкие непрерывные линии границ при подавлении шума.

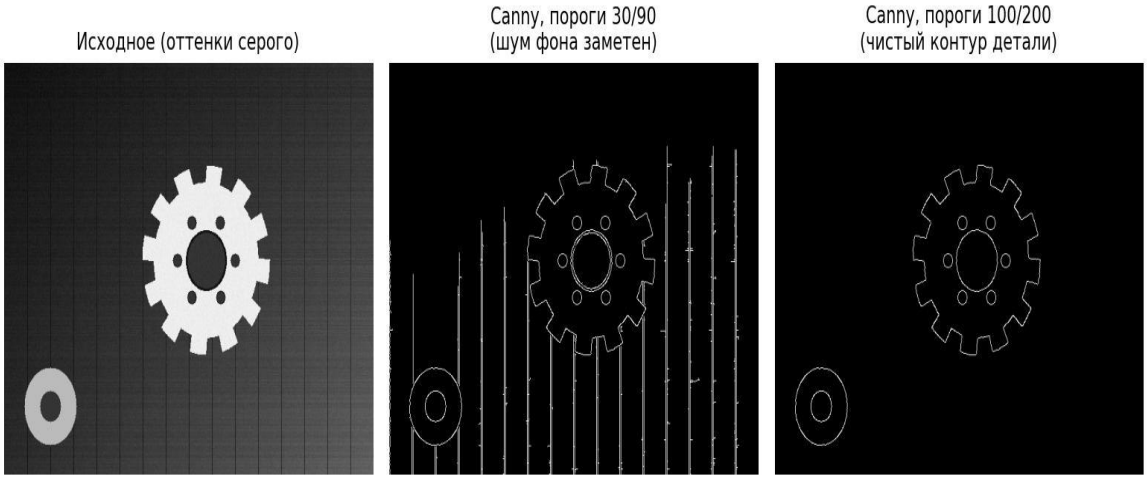


Рисунок 2 – Выделение границ детектором Кэнни при разных пороговых значениях: заниженные пороги пропускают шум фона (линии конвейера), завышенные – дают чистый контур объекта

После выделения границ выполняется поиск контуров – замкнутых или разомкнутых линий, ограничивающих связные области объекта. По контуру можно вычислить его площадь, периметр, ограничивающий прямоугольник (bounding box), выпуклую оболочку и степень отклонения формы контура от простых геометрических фигур. Эти характеристики часто оказываются достаточными для отбраковки деталей по форме на производстве без применения нейронной сети.

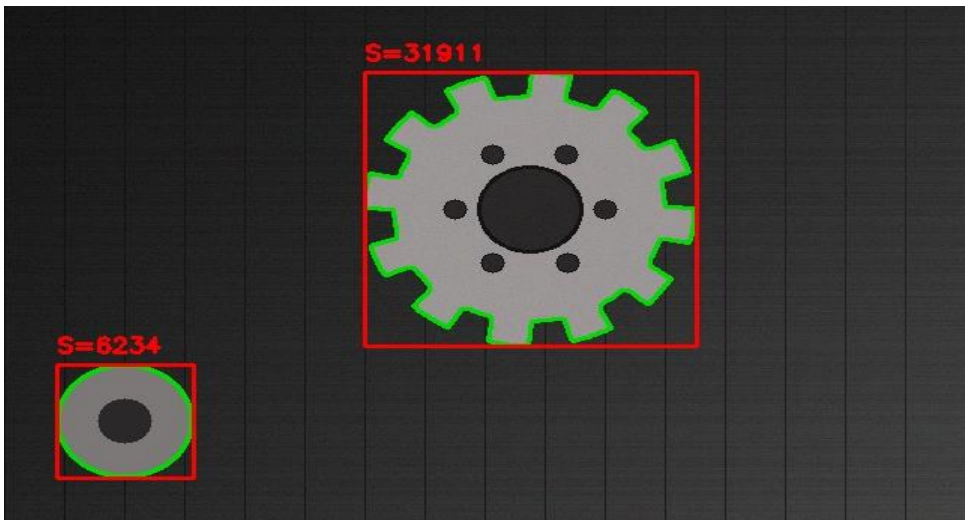


Рисунок 3 – Найденные контуры объектов с ограничивающими прямоугольниками (bounding box) и вычисленной площадью S в пикселях

Граница контура на рисунке 3 выделена зелёной линией – она проходит непосредственно по пикселям объекта. Красный прямоугольник – это ограничивающая рамка (bounding box), построенная уже по зелёному контуру как минимальный прямоугольник, в который контур помещается целиком; площадь S считается именно по контуру, а не по площади рамки.

Ключевые точки и локальные дескрипторы

Ключевой точкой (особой точкой) называется точка изображения, обладающая свойством устойчиво обнаруживаться при изменении масштаба, поворота и освещения кадра. К таким точкам относятся углы объекта, места пересечения линий, характерные элементы текстуры. Для их обнаружения используются детекторы Харриса (Harris), FAST, а также комбинированные алгоритмы ORB, SIFT и SURF, которые помимо самой точки строят её дескриптор – числовой вектор, описывающий локальную окрестность точки.

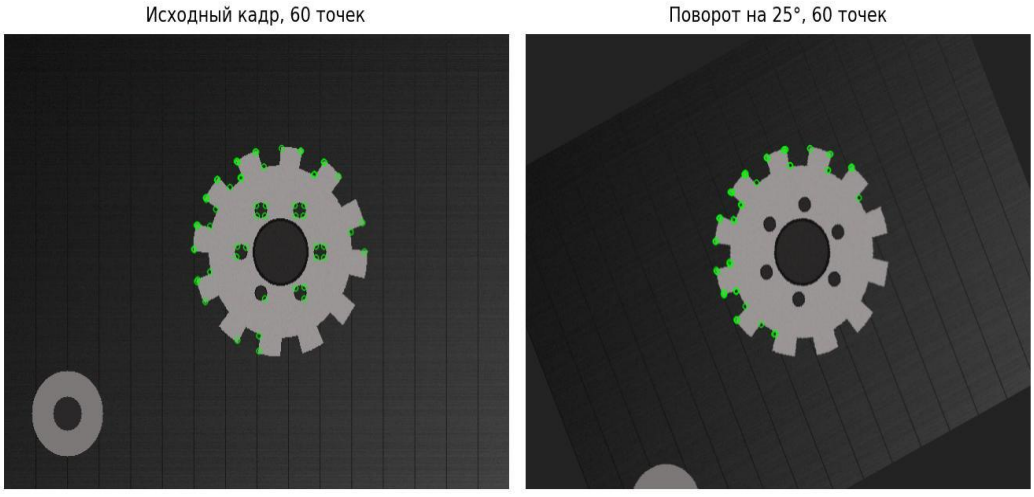


Рисунок 4 – Ключевые точки ORB на исходном кадре и на кадре, повёрнутом на 25°: большинство точек устойчиво повторяется на зубьях и отверстиях детали

Дескрипторы ключевых точек применяются для сопоставления одного и того же объекта на разных кадрах видеопотока (например, слежение за деталью на конвейере), для поиска эталонного шаблона на изображении и для оценки взаимного смещения кадров при вибрации камеры.

Цветовые и текстурные признаки

Если форма объекта постоянна, но задачей является отбраковка по цвету или качеству поверхности, используются цветовые и текстурные признаки. К цветовым относятся среднее значение и дисперсия каждого канала в цветовом пространстве RGB или HSV, а также цветовые гистограммы. К текстурным – характеристики локальной изменчивости яркости, в том числе получаемые с помощью матрицы совместной встречаемости уровней серого (GLCM) и метода локальных бинарных шаблонов (LBP), а также гистограммы направленных градиентов (HOG), которые описывают распределение направлений перепадов яркости в окрестности точки.

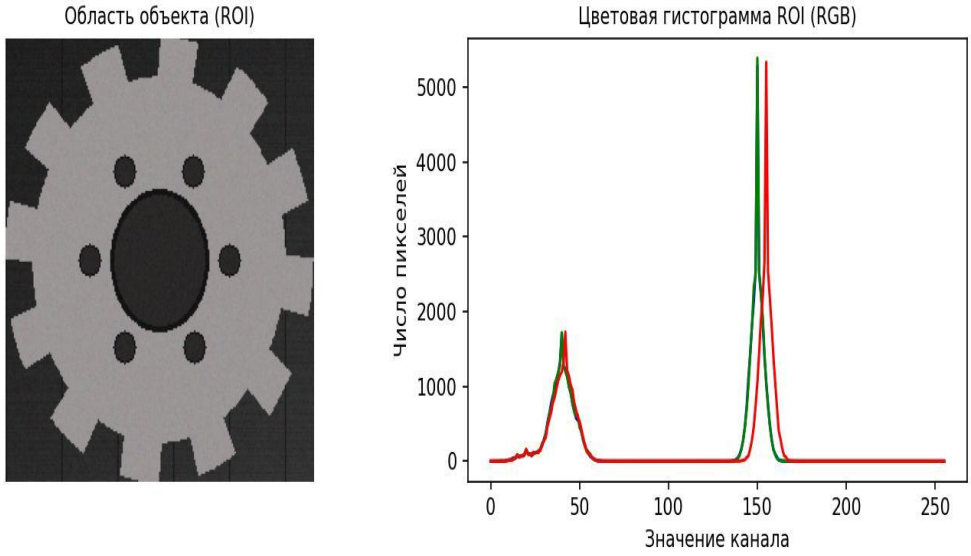


Рисунок 5 – Область объекта (ROI) и её цветовая гистограмма по каналам R, G, B

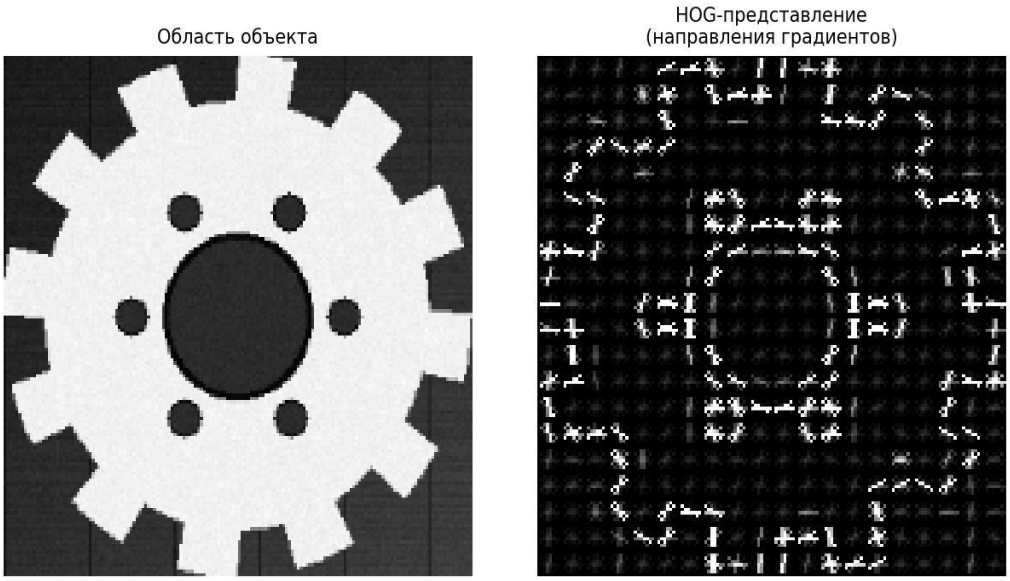


Рисунок 6 – Область объекта и её HOG-представление: направление штриха показывает преобладающее направление градиента яркости в каждой ячейке

Данные, в которых содержится информация о целевом признаке, называются размеченными. Прежде чем приступать к разметке большого набора изображений вручную (лабораторная работа №5) и последующему обучению нейронной сети (лабораторная работа №6), полезно выполнить предварительный анализ признаков и, при необходимости, простую сегментацию объекта от фона средствами классических алгоритмов обработки изображений – без привлечения нейронной сети. Такие алгоритмы рассматриваются в следующем разделе; каждый студент реализует один из них в соответствии со своим вариантом задания.

Алгоритмы сегментации объекта от фона (индивидуальное задание)

Помимо анализа локальных признаков (границ, ключевых точек, цвета и текстуры), на практике для выделения объекта на изображении часто применяются простые алгоритмы сегментации, не требующие обучения нейронной сети. В настоящей работе рассматриваются три таких алгоритма: заливка связной области, бинаризация с плавающим (перебираемым) порогом и бинаризация методом Оцу. Каждый студент реализует только один алгоритм – в соответствии с номером своего варианта, указанного в разделе «Варианты заданий».

Алгоритм заливки (Flood Fill)

Заливка – это процесс формирования сегмента из смежных пикселей одного цвета (или близких по цвету в пределах заданного допуска). Алгоритм подходит для выделения однородных по яркости объектов на относительно равномерном фоне.

В блоке начальных установок выполняется загрузка изображения, создаётся его карта данных (двумерный массив, каждая клетка которого соответствует одному пикселю изображения и хранит отметку о том, просмотрен ли уже этот пиксель), задаётся начальная точка заливки – произвольно выбранный пиксель внутри интересующего объекта – и организуется очередь, в которую записываются координаты ещё не обработанных пикселей сегмента.

Основной блок алгоритма образует цикл: на каждом шаге из головы очереди извлекается очередной пиксель, проверяются его четыре смежных соседа (сверху, снизу, слева, справа). Если цвет соседнего пикселя совпадает с цветом рассматриваемого (с учётом допустимого отклонения) и этот сосед ещё не был отмечен в карте данных, то его координаты добавляются в конец очереди, а в карте данных ставится отметка о просмотре. Цикл завершается, когда очередь пуста – это означает, что все пиксели связного однородного сегмента найдены и отмечены.

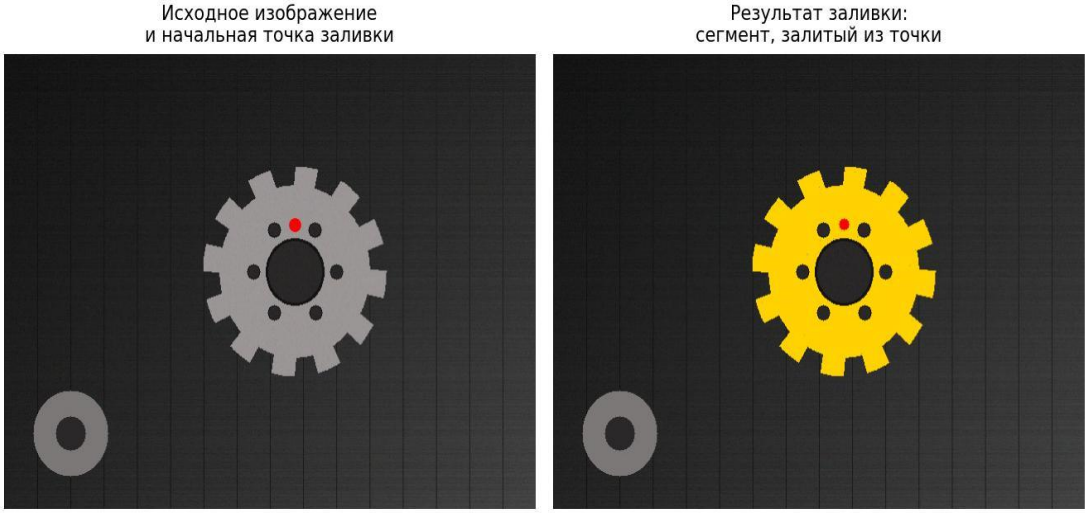


Рисунок 7 – Заливка связной области: слева – исходное изображение с начальной точкой внутри объекта, справа – результат заливки (выделенный сегмент)

Листинг ниже показывает загрузку изображения и пример работы с массивом координат пикселя и его цветом (базовые операции, используемые далее при реализации алгоритма на Python средствами библиотеки OpenCV):

```
import cv2
import numpy as np

img = cv2.imread('gear.png')
h, w = img.shape[:2]
```

```
mask = np.zeros((h + 2, w + 2), np.uint8)  # маска для floodFill
seed_point = (320, 180)  # начальная точка внутри объекта

# допустимое отклонение цвета соседнего пикселя от заливаемого
lo_diff = (10, 10, 10)
up_diff = (10, 10, 10)

cv2.floodFill(
   img, mask, seed_point, (0, 210, 255), lo_diff, up_diff)
cv2.imshow('Результат заливки', img)
cv2.waitKey(0)
```

Функция cv2.floodFill выполняет описанный выше алгоритм внутренними средствами библиотеки: она принимает изображение, вспомогательную маску, начальную точку и цвет заливки, а также допустимые отклонения по каждому цветовому каналу. Студенту, выполняющему данный вариант задания, необходимо не просто вызвать готовую функцию, а дополнительно реализовать её ключевой шаг – проверку одного из четырёх смежных пикселей (например, нижнего) – самостоятельно, в виде отдельной функции на Python, по аналогии с псевдокодом:

```
def check_bottom_neighbor(img_field, img, row, col, n_rows):
   # проверка нижнего смежного пикселя на принадлежность к сегменту
   if row + 1 >= n_rows:
      return None  # вышли за границу изображения
   color_cur = img[row, col]
   color_nb = img[row + 1, col]
   visited = img_field[row + 1][col] == 1
   same_color = tuple(color_cur) == tuple(color_nb)
   if same_color and not visited:
      img_field[row + 1][col] = 1   # отметка о просмотре
      return (row + 1, col)         # координаты в очередь
   return None
```

Самостоятельно необходимо реализовать аналогичные проверки для левого, правого и верхнего смежных пикселей, а также собрать из них основной цикл алгоритма (аналог while с очередью) и сравнить полученный «ручной» результат с результатом встроенной функции cv2.floodFill.

Бинаризация с плавающим порогом

Простейший способ отделить объект от фона – задать порог яркости t, относительно которого все пиксели полутонового изображения разделяются на два класса: пиксели темнее порога становятся чёрными (0), пиксели ярче порога – белыми (255). Такая операция называется бинаризацией.

Перевод цветного изображения в оттенки серого выполняется по формуле, учитывающей разную чувствительность человеческого глаза к красному, зелёному и синему цветам:

$$\mathbf{Y = 0{,}2126 \cdot R + 0{,}7152 \cdot G + 0{,}0722 \cdot B}$$

Название «плавающий порог» означает, что вместо однократного выбора одного значения t порог последовательно перебирается в некотором диапазоне (например, от 1 до 255), и для каждого значения формируется своё бинарное изображение. Это позволяет наглядно проследить, как меняется результат сегментации в зависимости от выбранного порога, и подобрать значение, при котором объект отделяется от фона наиболее чисто.

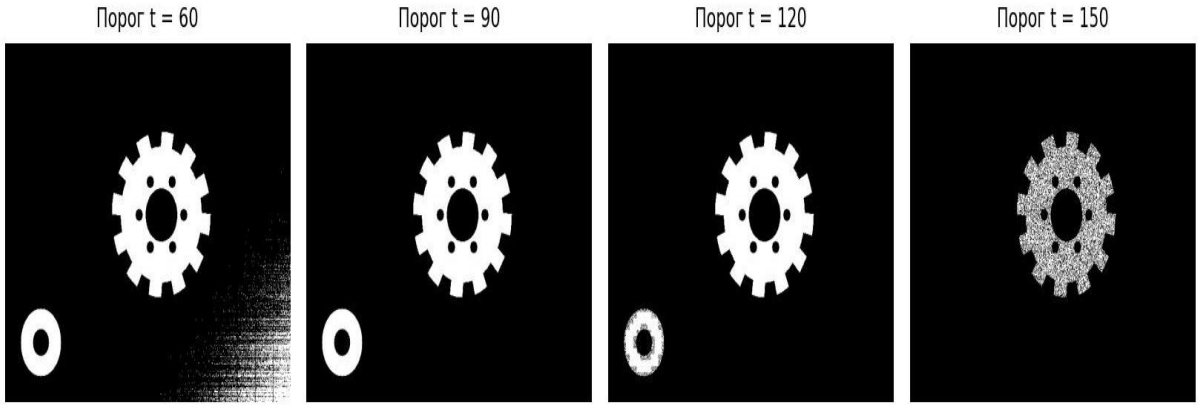


Рисунок 8 – Результат бинаризации при разных значениях порога t: при заниженном пороге в объект попадает часть светлого фона, при завышенном – объект рассыпается из-за шума

Алгоритм реализуется тройным вложенным циклом: во внешнем цикле перебираются значения порога t от 1 до 255; во внутренних двух циклах выполняется проход по всем строкам и столбцам изображения и сравнение яркости каждого пикселя с текущим порогом.

```
import cv2
import numpy as np
```

```
img = cv2.imread('gear.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
h, w = gray.shape

results = {}
for t in range(1, 256):        # перебор («плавание») порога
    bw = np.zeros((h, w), dtype=np.uint8)
    for i in range(h):
        for j in range(w):
            if gray[i, j] <= t:
                bw[i, j] = 0
            else:
                bw[i, j] = 255
    results[t] = bw
    # при необходимости сохранить bw в файл,
    # имя которого содержит значение порога t
    # cv2.imwrite(f'bw_{t}.png', bw)
```

В реальной программе тройной цикл по всем 255 значениям порога выполняется медленно; для учебных целей студенту достаточно рассчитать и сохранить результат для 4–5 характерных значений порога (например, 60, 90, 120, 150) и сравнить их между собой, как показано на рисунке 8, сделав вывод о наиболее подходящем значении для конкретного изображения.

Бинаризация методом Оцу

Метод Оцу – это гистограммный метод автоматического выбора порога бинаризации полутонового изображения. Идея метода заключается в том, чтобы разделить все пиксели изображения на два класса (фон и объект) так, чтобы внутриклассовая дисперсия яркости была минимальной (что эквивалентно максимуму межклассовой дисперсии). В отличие от плавающего порога, где значение t подбирается визуально самим исследователем, метод Оцу находит оптимальный порог автоматически, анализируя форму гистограммы яркости.

Межклассовая дисперсия для порога t вычисляется по формуле:

$$\sigma^2(t) = w_1(t) \cdot w_2(t) \cdot [m_1(t) - m_2(t)]^2$$

где t – испытываемое значение порога яркости; $w_1(t)$ и $w_2(t)$ – доли (веса) пикселей первого и второго класса соответственно; $m_1(t)$ и $m_2(t)$ – средние значения яркости пикселей первого и второго класса. Оптимальным считается порог t, при котором величина $\sigma^2(t)$ достигает максимума.

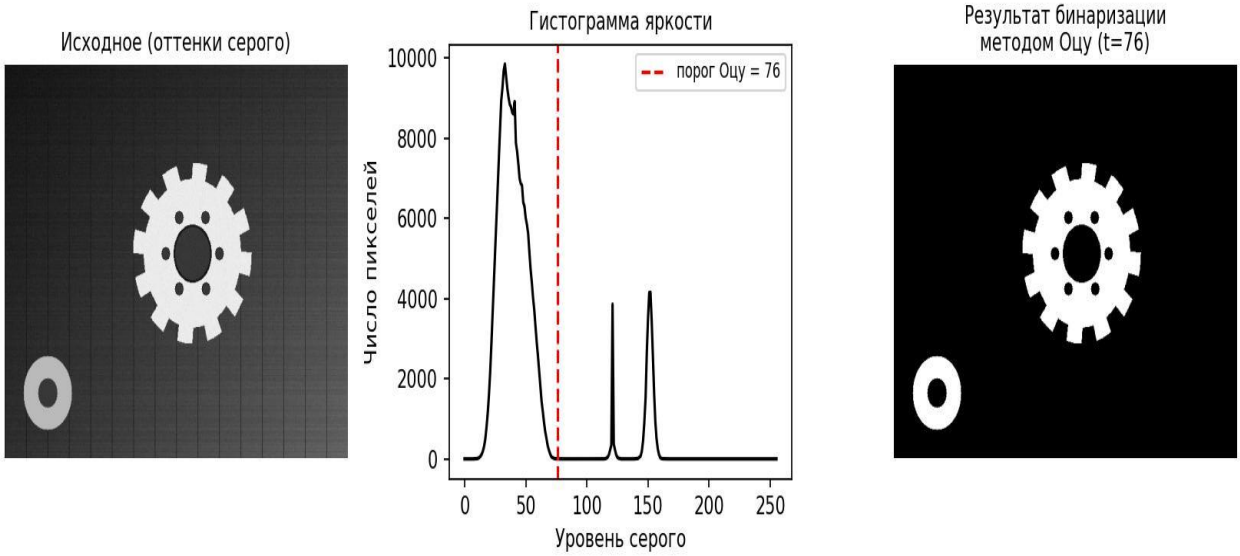


Рисунок 9 – Гистограмма яркости изображения с двумя выраженными пиками (фон и объект) и найденный методом Оцу порог бинаризации

Алгоритм реализуется следующим образом: сначала строится гистограмма яркости изображения – массив из 256 значений, в котором i-я ячейка содержит число пикселей с яркостью i. Затем в цикле по всем возможным значениям порога t от 1 до 255 вычисляются вес и средняя яркость каждого из двух классов, а по ним – межклассовая дисперсия $\sigma^2(t)$; запоминается максимальное найденное значение дисперсии и соответствующий ему порог.

```
import cv2
import numpy as np

img = cv2.imread('gear.png')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
hist = hist.flatten()
total = gray.size

# суммарная яркость всего изображения
sum_all = np.dot(np.arange(256), hist)
w1, sum1 = 0.0, 0.0
best_t, best_var = 0, -1.0

for t in range(256):
    w1 += hist[t]            # вес класса «фон»
```

```
    if w1 == 0:
        continue
    w2 = total - w1            # вес класса «объект»
    if w2 == 0:
        break
    sum1 += t * hist[t]
    m1 = sum1 / w1                    # среднее 1-го класса
    m2 = (sum_all - sum1) / w2        # среднее 2-го класса
    between_var = w1 * w2 * (m1 - m2) ** 2
    if between_var > best_var:
        best_var = between_var
        best_t = t

print('Оптимальный порог Оцу:', best_t)
_, bw_otsu = cv2.threshold(gray, best_t, 255, cv2.THRESH_BINARY)
```

После нахождения оптимального порога выполняется обычная бинаризация изображения по этому порогу, как в разделе «Бинаризация с плавающим порогом». Студенту, выполняющему данный вариант, необходимо реализовать вычисление порога самостоятельно (по формуле выше, без использования готового флага cv2.THRESH_OTSU) и лишь для проверки правильности сравнить полученное значение порога со значением, которое возвращает встроенная функция OpenCV с флагом THRESH_OTSU.

Достоинством метода Оцу является полностью автоматическая кластеризация пикселей на два класса без участия оператора и точное разделение объекта и фона при условии, что их яркости достаточно различаются (гистограмма имеет два выраженных пика, как на рисунке 9). Недостатком классического метода является то, что он рассчитан только на разделение изображения на два класса; для сегментации на большее число классов применяется существенно более сложный по вычислительной нагрузке мультипороговый метод Оцу.

Порядок выполнения работы

Работа состоит из общей части (выполняется всеми студентами одинаково) и индивидуальной части (выполняется в соответствии с номером варианта). Все шаги рекомендуется выполнять на одном и том же производственном объекте и одном и том же исходном наборе изображений, подготовленном в лабораторной работе №3.

Общая часть

Шаг 1. Подготовка программной среды. Подключите необходимые библиотеки Python и загрузите одно или несколько обработанных изображений из результатов лабораторной работы №3:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Шаг 2. Выделение границ методом Кэнни. Переведите изображение в оттенки серого и примените детектор Кэнни с двумя разными парами пороговых значений. Сравните результат и выберите пороги, при которых контур объекта выделяется непрерывно, а шум фона подавлен, как показано на рисунке 2.

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges1 = cv2.Canny(gray, 50, 150)
edges2 = cv2.Canny(gray, 100, 200)
```

Шаг 3. Поиск и анализ контуров. Найдите контуры объекта, вычислите для каждого его площадь, периметр и ограничивающий прямоугольник; отфильтруйте контуры по площади, отбросив шум:

```
_, obj_mask = cv2.threshold(gray, 95, 255, cv2.THRESH_BINARY)
contours, _ = cv2.findContours(
   obj_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for c in contours:
   area = cv2.contourArea(c)
   if area < 300:
      continue
   x, y, w, h = cv2.boundingRect(c)
   print('Площадь:', area, 'BBox:', (x, y, w, h))
```

Шаг 4. Обнаружение ключевых точек. Примените детектор ORB для нахождения ключевых точек и оцените их устойчивость на повернутом или сдвинутом изображении (рисунок 4):

```
orb = cv2.ORB_create(nfeatures=60)
keypoints, descriptors = orb.detectAndCompute(gray, None)
print('Найдено ключевых точек:', len(keypoints))
```

Шаг 5. Анализ цветовых и текстурных признаков. Вычислите цветовую гистограмму области объекта и постройте её HOG-представление (рисунки 5 и 6):

```
x, y, w, h = cv2.boundingRect(c)
```

```
roi = img[y:y+h, x:x+w]
hist_b = cv2.calcHist([roi], [0], None, [256], [0, 256])
```

По результатам шагов 1–5 сформируйте перечень признаков объекта, которые будут использоваться при последующей разметке изображений в лабораторной работе №5.

Индивидуальная часть (по варианту)

Шаг 6. В зависимости от номера своего варианта (раздел «Варианты заданий») реализуйте один из трёх алгоритмов сегментации объекта от фона:

- варианты 1–4 – алгоритм заливки (Flood Fill): найдите на изображении подходящую начальную точку внутри объекта, самостоятельно запрограммируйте проверку одного из смежных пикселей (по своему варианту – верхнего, нижнего, левого или правого), после чего сравните собственную реализацию с результатом функции cv2.floodFill;
- варианты 5–7 – бинаризация с плавающим порогом: рассчитайте результат бинаризации для 4–5 значений порога в указанном для варианта диапазоне, сравните полученные изображения между собой и выберите оптимальное значение порога для данного объекта;
- варианты 8–10 – метод Оцу: постройте гистограмму яркости, самостоятельно реализуйте цикл вычисления межклассовой дисперсии и поиска оптимального порога по приведённой формуле, сравните полученное значение порога со значением, которое возвращает встроенная функция cv2.threshold с флагом THRESH_OTSU.

Шаг 7. Сравните результат работы своего алгоритма (шаг 6) с контуром объекта, найденным в общей части работы (шаг 3): совпадает ли площадь и форма выделенной области; какой из подходов (анализ контура по Кэнни или выбранный алгоритм сегментации) дал более точный результат для вашего объекта и почему.

Шаг 8. Сформируйте итоговый перечень признаков и подходов к сегментации, которые могут быть использованы при разметке данных для выбранного производственного объекта в лабораторной работе №5.

Содержание отчёта

1) Цель работы.
2) Вариант задания и описание используемого производственного объекта.
3) Исходное изображение (или кадр видео) с указанием его параметров.

4) Результаты выделения границ методом Кэнни при разных пороговых значениях, сравнение и обоснование выбора.
5) Результаты поиска контуров: изображение с выделенными контурами, таблица характеристик контуров (площадь, периметр, bounding box).
6) Результаты обнаружения ключевых точек, оценка их устойчивости при повороте/сдвиге изображения.
7) Результаты анализа цветовых и текстурных признаков (гистограммы, HOG).
8) Описание и листинг реализации индивидуального алгоритма сегментации (заливка / плавающий порог / метод Оцу) в соответствии с вариантом.
9) Изображения, иллюстрирующие результат работы индивидуального алгоритма.
10) Сравнение результата индивидуального алгоритма с контуром, найденным в общей части работы.
11) Перечень признаков, выбранных для последующей разметки данных.
12) Выводы по результатам лабораторной работы.

Варианты заданий

Номер варианта определяется преподавателем либо выбирается студентом самостоятельно по согласованию с преподавателем. Общая часть работы (шаги 1–5) выполняется одинаково при любом варианте; индивидуальная часть (шаги 6–8) выполняется в соответствии со столбцом «Индивидуальный алгоритм» приведённой ниже таблицы.

№ варианта	**Производственный объект**	**Индивидуальный алгоритм (шаги 6–8)**
1	Деталь на конвейере (шестерня, болт, шайба)	Заливка: проверка нижнего смежного пикселя
2	Печатная плата или электронный компонент	Заливка: проверка верхнего смежного пикселя
3	Упаковка или тара (коробка, бутылка, банка)	Заливка: проверка левого смежного пикселя
4	Маркировка или этикетка на изделии	Заливка: проверка правого смежного пикселя
5	Металлическая поверхность с возможными дефектами	Плавающий порог, диапазон t = 30…90

№ варианта	Производственный объект	Индивидуальный алгоритм (шаги 6–8)
6	Пищевой продукт на конвейере	Плавающий порог, диапазон t = 60…120
7	Сварной шов или элемент конструкции	Плавающий порог, диапазон t = 90…150
8	Текстильное полотно или ткань	Метод Оцу с проверкой через THRESH_OTSU
9	Инструмент или крепёжное изделие на складе	Метод Оцу с построением графика $\sigma^2(t)$
10	Автомобильная деталь (диск, фара, элемент кузова)	Метод Оцу с сравнением на 2 изображениях

Допускается использовать тот же объект, что и в лабораторных работах №3 и предыдущей версии данной работы; при этом индивидуальный алгоритм сегментации выбирается строго по номеру варианта.

Контрольные вопросы

1) Что понимается под признаком объекта на изображении и для чего он используется?

2) Какие три основные задачи компьютерного зрения выделяют и чем они отличаются друг от друга?

3) Какие этапы включает алгоритм выделения границ методом Кэнни?

4) Что такое ключевая (особая) точка изображения и какими свойствами она должна обладать?

5) Чем дескриптор ключевой точки отличается от самой точки?

6) Что показывает гистограмма направленных градиентов (HOG) и для чего она применяется?

7) Чем отличается ограничивающий прямоугольник (bounding box), найденный по контуру, от самого контура?

8) Почему предварительный анализ признаков важен перед этапом разметки большого набора данных?

Вопросы 9–11 – по индивидуальному алгоритму (в соответствии со своим вариантом):

9) (Заливка) Что такое карта данных в алгоритме заливки и для чего она используется? Что служит условием окончания основного цикла алгоритма?

10) (Плавающий порог) Почему при слишком низком пороге в объект попадает часть фона, а при слишком высоком – объект «рассыпается»? Как рассчитывается перевод цветного изображения в оттенки серого?

11) (Метод Оцу) Какую величину максимизирует (или какую минимизирует) метод Оцу при выборе порога? Почему классический метод Оцу применим только для разделения на два класса?

12) В чём принципиальное различие между сегментацией классическим алгоритмом (заливка / порог / Оцу) и сегментацией с помощью обученной нейронной сети?

13) В каких случаях задачу выделения объекта можно решить простыми методами сегментации без обучения нейронной сети, а в каких это невозможно?